```java
import javax.swing.*;
import javax.swing.table.DefaultTableCellRenderer;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.io.*;
import java.util.ArrayList;
import java.util.List;

public class AdminCurrentweekpage extends JFrame implements ActionListener {
    // Column headers representing days of the week
    private static final String[] COLUMN_HEADERS = {"Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"};
    // Row headers representing meal times
    private static final String[] ROW_HEADERS = {"Breakfast", "1st Lunch", "2nd Lunch (Downstairs)", "2nd Lunch (Stairs)", "Supper"};

    private JLabel titlelabel; // Title displayed at the top of the frame
    private JTable table; // JTable to display the schedule
    private DefaultTableModel tablemodel; // Table model for managing data
    private JButton saveButton; // Button to save changes to the schedule
    private String date; // Starting date of the current week

    public AdminCurrentweekpage(String date, Admin admin) {
        this.date = date;
        this.setExtendedState(JFrame.MAXIMIZED_BOTH);
        this.setTitle("Schedule Table");
        this.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
        this.setLayout(new BorderLayout());

        // Define folder and file names for the schedule
        String foldername = date + " week";
        String filename = foldername + "/" + date + " week final";
        File filefinal = new File(filename);
```

```java

        // Check if the file for the current week exists
        if (!filefinal.exists()) {
            JOptionPane.showMessageDialog(
                    this,
                    "Week not scheduled",
                    "Error",
                    JOptionPane.ERROR_MESSAGE
            );
            this.dispose();
            return;
        }

        // Load the schedule data from the file
        String[][] data = processFile(filefinal);

        // Create and style the title label
        titlelabel = new JLabel("Current week - Starting Date: " + date, JLabel.CENTER);
        titlelabel.setFont(new Font("Serif", Font.BOLD, 24));

        // Initialize the table model with the given data
        tablemodel = new DefaultTableModel(data.length, COLUMN_HEADERS.length + 1) {
            @Override
            public String getColumnName(int column) {
                return (column == 0) ? "Meal Time" : COLUMN_HEADERS[column - 1];
            }

            @Override
            public boolean isCellEditable(int row, int column) {
                // Allow admin to edit all cells except the first column (row headers)
                return column != 0;
            }
        };

        // Populate the table model with row and column data
        for (int i = 0; i < data.length; i++) {
            tablemodel.setValueAt(ROW_HEADERS[i], i, 0); // Set row headers
```

```java
            for (int j = 0; j < data[i].length; j++) {
                tablemodel.setValueAt(data[i][j], i, j + 1); // Set schedule values
            }
        }

        // Create and style the JTable
        table = new JTable(tablemodel);
        table.setRowHeight(50); // Set row height for better readability
        table.setFont(new Font("Serif", Font.PLAIN, 16)); // Set font for table content
        table.getTableHeader().setFont(new Font("Serif", Font.BOLD, 18)); // Set font for table headers

        // Center-align table cell content
        DefaultTableCellRenderer centerRenderer = new DefaultTableCellRenderer();
        centerRenderer.setHorizontalAlignment(JLabel.CENTER);
        table.setDefaultRenderer(Object.class, centerRenderer);

        // Add table to a scroll pane for scrolling functionality
        JScrollPane scrollPane = new JScrollPane(table);
        scrollPane.setPreferredSize(new Dimension(800, 600));

        // Create and configure the save button
        saveButton = new JButton("Save Changes");
        saveButton.setFont(new Font("Arial", Font.BOLD, 18));
        saveButton.addActionListener(this);

        // Add the save button to a panel
        JPanel buttonPanel = new JPanel();
        buttonPanel.add(saveButton);

        // Add components to the frame
        this.add(titlelabel, BorderLayout.NORTH); // Add title label to the top
        this.add(scrollPane, BorderLayout.CENTER); // Add table to the center
        this.add(buttonPanel, BorderLayout.SOUTH); // Add button panel to the bottom
```

```java
        this.setVisible(true); // Make the frame visible
    }

    /**
     * Reads the file and processes its content into a 2D array.
     *
     * @param file The file to process.
     * @return A 2D array containing the file's data.
     */
    private static String[][] processFile(File file) {
        List<String[]> lines = new ArrayList<>();
        try (BufferedReader reader = new BufferedReader(new FileReader(file))) {
            String line;
            while ((line = reader.readLine()) != null) {
                // Split each line by whitespace and add to the list
                lines.add(line.trim().split("\\s+"));
            }
        } catch (IOException e) {
            e.printStackTrace(); // Print stack trace if an error occurs
        }
        return lines.toArray(new String[0][]); // Convert the list to a 2D array
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        if (e.getSource() == saveButton) {
            // Define folder and file names for saving the schedule
            String foldername = date + " week";
            String filename = foldername + "/" + date + " week final";

            try (FileWriter writer = new FileWriter(filename)) {
                // Save the updated schedule data to the file
                for (int i = 0; i < ROW_HEADERS.length; i++) {
                    for (int j = 1; j <= COLUMN_HEADERS.length; j++) {
                        writer.write(tablemodel.getValueAt(i, j)
```

```
+ " ");
                    }
                    writer.write("\n");
                }
                // Display a success message
                JOptionPane.showMessageDialog(this, "File saved successfully!", "Success", JOptionPane.INFORMATION_MESSAGE);
            } catch (IOException e1) {
                e1.printStackTrace(); // Print stack trace if an error occurs
            }
        }
    }
}
```

```java
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.time.DayOfWeek;
import java.time.LocalDate;

import javax.swing.*;

//whole class similar to the ChooseWeekViewPage for User but for admin instead
public class AdminChooseWeekViewPage extends JFrame implements ActionListener{
    private Admin admin;
    private JButton currentWeekButton;
    private JButton nextWeekButton;
    private JButton goBackButton;
    private JPanel buttonpanel;


    public AdminChooseWeekViewPage(Admin admin) {
        this.admin = admin;
        this.setDefaultCloseOperation(EXIT_ON_CLOSE);
        this.setLayout(new BorderLayout());
        this.setExtendedState(JFrame.MAXIMIZED_BOTH);
        this.setTitle("Choose Week");

        JLabel titleLabel = new JLabel("Choose a Week to View", JLabel.CENTER);
        titleLabel.setFont(new Font("Serif", Font.BOLD, 24));

        goBackButton = new JButton("Go Back");
        goBackButton.setFont(new Font("Arial", Font.PLAIN, 16));
        goBackButton.addActionListener(this);


        JPanel topPanel = new JPanel(new BorderLayout());
        topPanel.add(goBackButton, BorderLayout.WEST);
        topPanel.add(titleLabel, BorderLayout.CENTER);


        currentWeekButton = new JButton("Current Week");
        currentWeekButton.setFont(new Font("Arial", Font.BOLD,
```

```
18));
        currentWeekButton.setPreferredSize(new Dimension(200, 80));
        currentWeekButton.addActionListener(this);


        nextWeekButton = new JButton("Next Week");
        nextWeekButton.setFont(new Font("Arial", Font.BOLD, 18));
        nextWeekButton.setPreferredSize(new Dimension(200, 80));
        nextWeekButton.addActionListener(this);

        buttonpanel = new JPanel();
        buttonpanel.add(currentWeekButton);
        buttonpanel.add(nextWeekButton);

        this.add(topPanel, BorderLayout.NORTH);
        this.add(buttonpanel, BorderLayout.CENTER);

        this.setVisible(true);
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        if(e.getSource() == goBackButton) {
            this.dispose();
            new AdminMainFrame(admin);
        }
        if(e.getSource() == currentWeekButton) {
            LocalDate today = LocalDate.now();
            DayOfWeek dayinweek = today.getDayOfWeek();
            int daynumber = dayinweek.getValue();
            LocalDate startofthisweek = today.minusDays(daynumber - 1);

            String Startofcurrentweekdate = startofthisweek.toString();

            new AdminCurrentweekpage(Startofcurrentweekdate, admin);
        }
        if(e.getSource() == nextWeekButton) {
            LocalDate today = LocalDate.now();
```

```
            DayOfWeek dayinweek = today.getDayOfWeek();
            int daynumber = dayinweek.getValue();
            LocalDate startofnextweek = today.plusDays(7);
            startofnextweek = startofnextweek.minusDays(daynumber - 1);
            String Startofcurrentweekdate = startofnextweek.toString();

            new AdminCurrentweekpage(Startofcurrentweekdate, admin);
        }
    }

}

```

```java
import java.awt.BorderLayout;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.awt.Font;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;

import javax.swing.BorderFactory;
import javax.swing.Box;
import javax.swing.BoxLayout;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JPasswordField;
import javax.swing.JTextField;
import javax.swing.border.EmptyBorder;

public class AdminRegisterPage extends JFrame implements ActionListener {

    // File paths for storing admin email and password data
    private static final String Emaildocumentpath = "AdminEmail.txt";
    private static final String Passworddocumentpath = "Adminpassword.txt";

    // UI components
    JLabel Register; // Label for the registration title
    JLabel Emaillabel; // Label for the email field
    JLabel codelabel; // Label for the password field
    JButton Registerbutton; // Button to register a new admin
    JTextField Email; // Text field for entering the email
    JPasswordField PasswordEnter; // Password field for entering the password
    JPanel textfieldpanel; // Panel to hold the input fields
    JPanel SouthPanel; // Panel to hold the register button
    Admin admin; // Reference to the Admin object
```

```java

    AdminRegisterPage(Admin admin) {
        this.admin = admin; // Store the passed Admin object
        this.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE); // Close only this window on exit
        this.setLayout(new BorderLayout()); // Use BorderLayout for the frame
        this.setExtendedState(JFrame.MAXIMIZED_BOTH); // Maximize the frame
        this.setTitle("Register"); // Set the frame title

        // Panel to hold the Register button
        SouthPanel = new JPanel();
        SouthPanel.setLayout(new FlowLayout(FlowLayout.CENTER, 10, 150));

        // Title label for the registration page
        Register = new JLabel("Register", JLabel.CENTER);
        Register.setFont(new Font("Serif", Font.BOLD, 50));
        Register.setBorder(BorderFactory.createEmptyBorder(100, 0, 0, 0)); // Add padding around the label

        // Panel to hold the email and password fields
        textfieldpanel = new JPanel();
        textfieldpanel.setLayout(new BoxLayout(textfieldpanel, BoxLayout.Y_AXIS));
        textfieldpanel.setBorder(new EmptyBorder(100, 200, 20, 100)); // Add padding around the panel

        // Email field and its label
        Emaillabel = new JLabel("Email: ");
        Emaillabel.setPreferredSize(new Dimension(100, 40));
        Email = new JTextField();
        Email.setPreferredSize(new Dimension(2000, 50));
        Email.setMaximumSize(new Dimension(2000, 50));

        // Password field and its label
        codelabel = new JLabel("Password: ");
        codelabel.setPreferredSize(new Dimension(100, 40));
        PasswordEnter = new JPasswordField();
        PasswordEnter.setEchoChar('*'); // Mask the entered characters
```

```java
        PasswordEnter.setPreferredSize(new Dimension(2000, 50));
        PasswordEnter.setMaximumSize(new Dimension(2000, 50));

        // Register button configuration
        Registerbutton = new JButton("Register");
        Registerbutton.setLayout(new FlowLayout(FlowLayout.CENTER));
        Registerbutton.setPreferredSize(new Dimension(400, 80));
        Registerbutton.addActionListener(this); // Add action listener for the button

        // Add components to the input panel
        textfieldpanel.add(Emaillabel);
        textfieldpanel.add(Box.createVerticalStrut(5)); // Add spacing between components
        textfieldpanel.add(Email);
        textfieldpanel.add(Box.createVerticalStrut(20));
        textfieldpanel.add(codelabel);
        textfieldpanel.add(Box.createVerticalStrut(5));
        textfieldpanel.add(PasswordEnter);

        // Add Register button to the south panel
        SouthPanel.add(Registerbutton);

        // Add all panels to the frame
        this.add(textfieldpanel, BorderLayout.CENTER);
        this.add(Register, BorderLayout.NORTH);
        this.add(SouthPanel, BorderLayout.SOUTH);
        this.setVisible(true); // Make the frame visible
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        if (e.getSource() == Registerbutton) {
            // Get the email and password entered by the user
            String registerEmail = Email.getText();
            char[] passwordchararray = PasswordEnter.getPassword();
            String password = new String(passwordchararray); // Convert password to String

            // Write the email to the file
```

```java
            try (BufferedWriter emailWriter = new BufferedWriter (new FileWriter(Emaildocumentpath, true))) {
                emailWriter.write(registerEmail + "|"); // Separate entries with a delimiter
                emailWriter.newLine();
            } catch (IOException e1) {
                e1.printStackTrace(); // Print error in case of an exception
            }

            // Write the password to the file
            try (BufferedWriter passwordWriter = new BufferedWriter(new FileWriter(Passworddocumentpath, true))) {
                passwordWriter.write(password + "|"); // Separate entries with a delimiter
                passwordWriter.newLine();
            } catch (IOException e1) {
                e1.printStackTrace(); // Print error in case of an exception
            }

            // Close the registration page and create a new Admin object
            this.dispose();
            new Admin(registerEmail, password);
        }
    }
}
```

```java
import java.awt.BorderLayout;
import java.awt.Dimension;
import java.awt.Font;
import java.awt.GridBagConstraints;
import java.awt.GridBagLayout;
import java.awt.GridLayout;
import java.awt.Insets;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

import java.io.BufferedReader;
import java.io.File;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.time.DayOfWeek;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.PriorityQueue;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFileChooser;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.filechooser.FileNameExtensionFilter;

public class AdminMainFrame extends JFrame implements ActionListener{

    //initialize variables and Jcomponents
    Admin admin;
    JButton create_new_Admin;
    JButton View_current_week_schedule;
```

```java
    JButton Schedule_current_week;
    JLabel label;
    JPanel buttonpanel;
    JPanel southPanel;
    JButton Schedule_to_picture;

    //headers and

    private static final String[] COLUMN_HEADERS = {"","Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"};
    private static final String[] ROW_HEADERS = {"Breakfast", "1st Lunch", "2nd Lunch (Downstairs)", "2nd Lunch (Stairs)", "Supper"};

    AdminMainFrame(Admin admin){
        //initialize frame default options
        this.setDefaultCloseOperation(EXIT_ON_CLOSE);
        this.setLayout(new BorderLayout());
        this.setExtendedState(JFrame.MAXIMIZED_BOTH);
        this.setTitle("Welcome");

        this.admin = admin;
        String text = admin.getEmail();

        //header for frame
        label = new JLabel("Welcome! " + text, JLabel.CENTER);
        label.setFont(new Font("Serif", Font.BOLD, 50));
        label.setBorder(BorderFactory.createEmptyBorder(100, 0, 50, 0));

        //south panel
        southPanel = new JPanel(new GridBagLayout());
        GridBagConstraints gbc = new GridBagConstraints();
        gbc.insets = new Insets(10, 10, 10, 10);

        //button panel for buttons
        buttonpanel = new JPanel(new GridLayout(1, 3, 20, 0));
        buttonpanel.setPreferredSize(new Dimension(900,100));
        //button size
        Dimension buttonsize = new Dimension(250,80);

```

```java
        //define all buttons
        create_new_Admin = new JButton("create new Admin");
        create_new_Admin.addActionListener(this);
        create_new_Admin.setPreferredSize(buttonsize);
        View_current_week_schedule = new JButton("view current week schedule");
        View_current_week_schedule.addActionListener(this);
        View_current_week_schedule.setPreferredSize(buttonsize);
        Schedule_current_week = new JButton("Schedule current week");
        Schedule_current_week.setPreferredSize(buttonsize);
        Schedule_current_week.addActionListener(this);

        Schedule_to_picture = new JButton("Output schedule");
        Schedule_to_picture.setPreferredSize(buttonsize);
        Schedule_to_picture.addActionListener(this);


        //add components to the frame
        buttonpanel.add(create_new_Admin);
        buttonpanel.add(View_current_week_schedule);
        buttonpanel.add(Schedule_current_week);
        buttonpanel.add(Schedule_to_picture);

        southPanel.add(buttonpanel,gbc);

        this.add(label, BorderLayout.NORTH);
        this.add(southPanel, BorderLayout.CENTER);
        this.setVisible(true);
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        //admin click on schedule
        if(e.getSource() == Schedule_current_week) {
            //get current week (any date) and calculate start date
            LocalDate today = LocalDate.now();
            DayOfWeek dayinweek = today.getDayOfWeek();
            int daynumber = dayinweek.getValue();
            LocalDate startofweek = today.minusDays(daynumber - 1);
```

```
            String startofcurrentweek = startofweek.toString();
            //schedule (Figure 8.1)
            String[][] result = schedulefinalarray(startofcurrentweek);
            //display success schedulling
            JOptionPane.showMessageDialog(null,
                    "File scheduled successfully!", "Success", JOptionPane.INFORMATION_MESSAGE);
        }
        if(e.getSource() == View_current_week_schedule) {
            new AdminChooseWeekViewPage(admin);

        }
        if(e.getSource() == create_new_Admin) {
            new AdminRegisterPage(this.admin);
        }
        if(e.getSource() == Schedule_to_picture) {
            //similar to the start of week date calculation mentioned in Figure 2.1
            LocalDate today = LocalDate.now();
            DayOfWeek dayinweek = today.getDayOfWeek();
            int daynumber = dayinweek.getValue();
            LocalDate startofthisweek = today.minusDays(daynumber - 1);
            String date = startofthisweek.toString();
            String foldername = date + " week";
            String filename = foldername + "/" + date + " week final";
            File folder = new File(foldername);
            File file = new File(filename);
            //if final file exist(week scheduled)
            if(folder.exists() && file.exists()) {
                String[][] timetable = processFile(new File(filename));
                //create a StringBuilder from java.lang
                StringBuilder html = new StringBuilder();
                //a JFileChooser for Admin to save the HTML file
                JFileChooser fileChooser = new JFileChooser();
                fileChooser.setDialogTitle("Save");
                fileChooser.setFileFilter(new FileNameExtensionFilter("HTML Files", "html"));
                /*defines HTML document structure and styles the
```

```
table with boardered cells,
                 * padding and centered text*/
                html.append("<!DOCTYPE html>\n<html>\n<head>\n<title>Schedule</title>\n");
                html.append("<style>table {border-collapse: collapse; width: 50%;}");
                html.append("th, td {border: 1px solid black; padding: 8px; text-align: center;}</style>\n");
                html.append("</head>\n<body>\n");
                //Add title, row and column headers
                html.append("<h2>Schedule week " + date +" </h2>\n<table>\n");
                html.append("<tr>");
                for (String colHeader : COLUMN_HEADERS) {
                    html.append("<th>").append(colHeader).append("</th>");
                }
                html.append("</tr>\n");
                /*fill the table with data: this program iterate through each row.
                 * The first column contains meal time labels and remaining column contain
                 * scheduled data*/
                for (int i = 0; i < timetable.length; i++) {
                    html.append("<tr>");
                    html.append("<th>").append(ROW_HEADERS[i]).append("</th>");
                    for (String cell : timetable[i]) {
                        html.append("<td>").append(cell).append("</td>");
                    }
                    html.append("</tr>\n");
                }
                //close the html document by ending
                html.append("</table>\n</body>\n</html>");
                //open file dialogue
                int userSelection = fileChooser.showSaveDialog(null);
                if (userSelection == JFileChooser.APPROVE_OPTION) {
                    //Retrieves the files path chosen by user and change the file name with .html ending
```

```
                    File fileToSave = fileChooser.getSelectedFile();
                    String filePath = fileToSave.getAbsolutePath();
                    if (!filePath.toLowerCase().endsWith(".html")) {
                        filePath += ".html";
                    }
                    /*Uses Filewriter to write HTML content to the file
                     * and use try and catch to ensure file choose properly*/
                    try (FileWriter writer = new FileWriter(filePath)) {
                        writer.write(html.toString());
                    } catch (IOException e1) {
                        e1.printStackTrace();
                    }
                } else {
                    System.out.println("Save operation was canceled.");
                }

            }
            else {
                //display error when week is not scheduled yet
                JOptionPane.showOptionDialog(
                        null,
                        "Week not scheduled",
                        "error",
                        JOptionPane.DEFAULT_OPTION,
                        JOptionPane.ERROR_MESSAGE,
                        null,
                        new Object[] { "OK" },
                        "OK"
                    );
            }
        }
    }


    private String[][] schedulefinalarray(String
```

```
startOfWeekDate){
        //check if that week have already scheduled
        String filename = startOfWeekDate + " week final";
        String foldername = startOfWeekDate + " week";
        File folder = new File(foldername);
        File filefinal = new File(foldername + "/" + filename);
        if(!filefinal.exists()) {
            //if not scheduled
            //get all the file exists in the folder
            //and create a HashMap to store data for each email/Username
            File[] files = folder.listFiles();
            Map<String, String[][]> emailToDataMap = new HashMap<>();
            //loop through all files in the folder
            //if the file contains the start of week date then:
            //1. extract the email from name using extractEmailFromFileName() method
            //(in figure 8.2)
            //2. Read schedule data by user
            //3. Stores data into the HashMap
            for (File file : files) {
                if (file.getName().contains(startOfWeekDate)) {
                    String email = extractEmailFromFileName(file.getName());
                    String[][] data = processFile(file);
                    emailToDataMap.put(email, data);
                }
            }
            //call organizeDataForLocations() method to combine into 1 final schedule
            //shown in figure 8.3
            String[][] finalArray = organizeDataForLocations(emailToDataMap);
            //write the final schedule 2D array into the final file and return it
            try(FileWriter writer = new FileWriter(filefinal)) {
                for(String[] row : finalArray) {
                    for(String cell : row) {
                        writer.write(cell + " ");
                    }
                    writer.write("\n");
```

```
                }
            } catch (IOException e1) {
                e1.printStackTrace();
            }
            return finalArray;
            }
        //if the file exist (already scheduled) then we don't have to reschedule
        //return the original schedulled data.
        else {
            String[][] finalArray = processFile(filefinal);
            return finalArray;
        }
    }

    private static String extractEmailFromFileName(String fileName) {
        /*define a regular expression:
         * ^ means start of string
         * (.*?) means capture the shortest sequence of any characters
         * (.*?) and \s+ ends at the first space (one or more spaces)*/
        Pattern pattern = Pattern.compile("^(.*?)\\s+");
        //matcher to apply pattern to filename
        Matcher matcher = pattern.matcher(fileName);
        //return everything before 1st space
        if (matcher.find()) {
            return matcher.group(1);
        }
        return "";
    }

    private static String[][] processFile(File file) {
        List<String[]> lines = new ArrayList<>();
        try (BufferedReader reader = new BufferedReader(new FileReader(file))) {
            String line;
            while ((line = reader.readLine()) != null) {
                String[] columns = line.trim().split("\\s+");
                lines.add(columns);
            }
```

```java
        } catch (IOException e1) {
            e1.printStackTrace();
        }
        String[][] result = new String[lines.size()][];
        for (int i = 0; i < lines.size(); i++) {
            result[i] = lines.get(i);
        }
        return result;
    }

    private static String[][] organizeDataForLocations (Map<String, String[][]> emailToDataMap) {
        //determines the size of the schedule by iterating through each person's availability of data.
        int numRows = 0, numCols = 0;
        for (String[][] data : emailToDataMap.values()) {
            numRows = data.length;
            numCols = data[0].length;
        }
        //creates a final 2D array for final schedule starting everything with " "
        String[][] finalArray = new String[numRows][numCols];
        for (int i = 0; i < numRows; i++) {
            for (int j = 0; j < numCols; j++) {
                finalArray[i][j] = " ";
            }
        }
        //this map assignmentCount keep track of how many times each email is assigned and set to 0 initially
        //(Refer to Criterion A - Success Critera)
        Map<String, Integer> assignmentCount = new HashMap<>();
        for (String email : emailToDataMap.keySet()) {
            assignmentCount.put(email, 0);
        }
        //creates a Map<Integer, List<String>> to associate each location
        //(column index with the people available to that location)
        Map<Integer, List<String>> locationToEmails = new HashMap<>();
        for (Map.Entry<String, String[][]> entry : emailToDataMap.entrySet()) {
```

```
            String email = entry.getKey();
            String[][] data = entry.getValue();
            for (int j = 0; j < numCols; j++) {
                for (int i = 0; i < numRows; i++) {
                    if ("X".equals(data[i][j])) {
                        //if the cell is "X", the same "X" copied to the final array
                        //because it indicate that that location doesn't need to be scheduled
                        finalArray[i][j] = "X";
                    } else if ("Y".equals(data[i][j])) {
                        //if "Y" then the email is added to the locationToEmails:
                        //meaning that they are available at that specific location
                        locationToEmails.computeIfAbsent(j, k -> new ArrayList<>()).add(email);
                    }
                }
            }
        }
        /*convert locationToEmails into a list of entries and sort it by number of people available in
         * ascending order (ensures that locations with fewer available space are filled first, minimizing
         * schedulling conflicts)*/
        List<Map.Entry<Integer, List<String>>> sortedLocations = new ArrayList<>(locationToEmails.entrySet());
        sortedLocations.sort(Comparator.comparingInt(e -> e.getValue().size()));
        /*priority queue is used to distribute work evenly.
         * The queue is sorted based on assignmentCount,
         * ensuring that the least-assigned person is selected first*/
        PriorityQueue<String> emailQueue = new PriorityQueue<>(Comparator.comparingInt(assignmentCount::get));
        /*loop through each location in sorted order and gets the column index (locationindex)
         * and the list of people available for that location.
         * It also clears the emailQueue and refills it with available people for that location*/
        for (Map.Entry<Integer, List<String>> locationEntry :
```

```
sortedLocations) {
            int locationIndex = locationEntry.getKey();
            List<String> emailsAtLocation = locationEntry.getValue();
            emailQueue.clear();
            emailQueue.addAll(emailsAtLocation);
            //iterate through all rows to assign people at given location and skip location for "X"
            for (int i = 0; i < numRows; i++) {
                if (!"X".equals(finalArray[i][locationIndex])) {
                    /*Polls the least-assigned person from the queue and check if they are available
                     * at that specific location. If so, assign them into the final array and increase their
                     * assignment count. Breaks the loop once a person is assigned, and move onto the next row*/
                    while (!emailQueue.isEmpty()) {
                        String email = emailQueue.poll();
                        if ("Y".equals(emailToDataMap.get(email)[i][locationIndex])) {
                            finalArray[i][locationIndex] = email;
                            assignmentCount.put(email, assignmentCount.get(email) + 1);
                            break;
                        }
                    }
                    /*if no one is assigned in a location, this means that no one is available
                     * set the cell to "X" which enable Admin to edit that spot*/
                    if (finalArray[i][locationIndex].equals(" ")) {
                        finalArray[i][locationIndex] = "X";
                    }
                }
            }
        }
        //return the final Array
        return finalArray;
    }

```

```
}
```

```java
import javax.swing.*;
import javax.swing.table.DefaultTableCellRenderer;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.io.File;
import java.io.FileWriter;
import java.io.IOException;

public class Scheduletablepage extends JFrame implements ActionListener{
    private JTable scheduleTable;
    private DefaultTableModel tableModel;
    private JButton saveButton;
    private JPanel buttonpanel;
    private JPanel panel;
    private String[][] timeSlots;
    private String date;
    private int weekNumber;
    private String weekAB;
    private JLabel titlelabel;
    private JPanel titlepanel;
    private JLabel noticelabel;
    private JPanel noticepanel;
    private User user;

    public Scheduletablepage(String date, int weekNumber, String weekAB, User user) {
        this.user = user;
        this.date = date;
        this.weekNumber = weekNumber;
        this.weekAB = weekAB;
        this.timeSlots = new String[5][7];

        this.setExtendedState(JFrame.MAXIMIZED_BOTH);
        this.setTitle("Schedule Table");
        this.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
        this.setLayout(new BorderLayout());

        //initialize JLabel for heading of the page
        titlelabel = new JLabel("Schedule New Week - Week " +
```

```
weekNumber + " (" + weekAB + ") - Starting Date: " + date, JLabel.CENTER);
        titlelabel.setFont(new Font("Serif", Font.BOLD, 24));
        titlepanel = new JPanel(new BorderLayout());
        titlepanel.add(titlelabel, BorderLayout.CENTER);

        //initialize JLabel for notice
        noticelabel = new JLabel("Notice: Sunday only allows brunch and tea duty for teachers", JLabel.CENTER);
        noticelabel.setFont(new Font("Serif", Font.BOLD, 16));
        noticelabel.setForeground(Color.RED);
        noticepanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
        noticepanel.add(noticelabel);

        //row and column headers
        String[] days = {"Monday", "Tuesday", "Wednesday", "Thursday",
                "Friday", "Saturday", "Sunday"};
        String[] times = {"Breakfast", "1st Lunch", "2nd Lunch (Downstairs)",
                "2nd Lunch (Stairs)", "Supper"};
        timeSlots = new String[times.length][days.length];
        /*table model is initialized with rows corresponding to times.lengh
         * and columns corresponding to days.length+1
         * (additional column for time slot labels)*/
        tableModel = new DefaultTableModel(times.length, days.length + 1) {
            @Override
            public boolean isCellEditable(int row, int column) {
                if (column == 7 && (row == 0 || row == 2 || row == 3)) {
                    return false;
                }
                return false;
            }
        };
        //set column headers for table
        tableModel.setColumnIdentifiers(
                new String[]{"", "Monday", "Tuesday", "Wednesday",
```

```
                        "Thursday", "Friday", "Saturday", "Sunday"});
        /*Populates Jtable with initial values, setting time slots
         * int he first column and marking it "Available" across the grid*/
        for (int i = 0; i < times.length; i++) {
            tableModel.setValueAt(times[i], i, 0);
            for (int j = 1; j <= days.length; j++) {
                if (j == 7 && (i == 0 || i == 2 || i == 3)) {
                    tableModel.setValueAt("", i, j);
                } else {
                    tableModel.setValueAt("Available", i, j);
                }
            }
        }
        scheduleTable = new JTable(tableModel);
        scheduleTable.setRowHeight(80);
        //modify how data is displayed in table cells
        DefaultTableCellRenderer centerRenderer = new DefaultTableCellRenderer();
        centerRenderer.setHorizontalAlignment(JLabel.CENTER);
        //applies a custom cell renderer (refer to figure 4.1) to the table
        scheduleTable.setDefaultRenderer(Object.class, new CustomCellRenderer());
        //add a mouse listener when a user clicks on certain cell in the table
        scheduleTable.addMouseListener(new java.awt.event.MouseAdapter() {
            public void mousePressed(java.awt.event.MouseEvent evt) {
                int row = scheduleTable.rowAtPoint(evt.getPoint());
                int col = scheduleTable.columnAtPoint(evt.getPoint());
                /*prevent user from clicking extra time slot on Sunday*/
                if (col == 7 && (row == 0 || row == 2 || row == 3)) {
                    return;
                }
```

```
                /* Toggle between "Unavailable" and "Available" for user when they are clicking
                 * on the table representing their availability on that specific time slot
                 * (Refer to Criterion A - Success Criteria)
                 * (Refer to Criterion B - Design of Table)*/
                if (col > 0 && row >= 0) {
                    String currentValue = (String) tableModel.getValueAt(row, col);
                    if ("Available".equals(currentValue)) {
                        tableModel.setValueAt("Unavailable", row, col);
                    } else {
                        tableModel.setValueAt("Available", row, col);
                    }
                    scheduleTable.repaint();
                }
            }
        });


        saveButton = new JButton("Save");
        saveButton.setPreferredSize(new Dimension(400, 80));
        saveButton.addActionListener(this);

        panel = new JPanel(new FlowLayout(FlowLayout.CENTER));
        panel.add(saveButton);


        buttonpanel = new JPanel();
        buttonpanel.setLayout(new BoxLayout(buttonpanel, BoxLayout.Y_AXIS));
        buttonpanel.add(noticepanel);
        buttonpanel.add(Box.createVerticalStrut(10));
        buttonpanel.add(panel);

        this.setLayout(new BorderLayout());
        this.add(titlepanel, BorderLayout.NORTH);
        this.add(new JScrollPane(scheduleTable), BorderLayout.CENTER);
        this.add(buttonpanel, BorderLayout.SOUTH);
```

```java

        this.setVisible(true);
    }

    private class CustomCellRenderer extends DefaultTableCellRenderer {
        @Override//override the superclass to get default rendering for the cell
        public Component getTableCellRendererComponent(JTable table,
                Object value, boolean isSelected, boolean hasFocus,
                int row, int column) {
            Component cell = super.getTableCellRendererComponent(
                    table, value, isSelected, hasFocus, row, column);

            if (column == 7 && (row == 0 || row == 2 || row == 3)) {
                //if sunday and unchangeable time slot set it always to grey
                cell.setBackground(Color.LIGHT_GRAY);
                cell.setForeground(Color.DARK_GRAY);
            } else if ("Available".equals(value)) {
                //if available turn green
                cell.setBackground(Color.GREEN);
                cell.setForeground(Color.BLACK);
            } else if ("Unavailable".equals(value)) {
                //if unavailable turn red
                cell.setBackground(Color.RED);
                cell.setForeground(Color.WHITE);
            } else {
                //default colour
                cell.setBackground(Color.WHITE);
                cell.setForeground(Color.BLACK);
            }
            return cell;
        }
    }


```

```java
    @Override
    public void actionPerformed(ActionEvent e) {
        if(e.getSource() == saveButton) {
            String[][] result = new String[5][7];
            for (int i = 0; i < timeSlots.length; i++) {
                for (int j = 0; j < 7; j++) {
                    if(scheduleTable.getValueAt(i,j+1).equals("Unavailable")) {
                        //if user unavailable, put 'N' in the txt file
                        result[i][j] = "N";
                    }
                    else if(scheduleTable.getValueAt(i,j+1).equals("Available")){
                        //if user available, put 'Y'
                        result[i][j] = "Y";
                    }
                    else {
                        //else uneditable, put 'X'
                        result[i][j] = "X";
                    }
                }
            }
            //set folder and file names
            String foldername = date + " week";
            String filename = user.getEmail() + " " + date + " week";
            File folder = new File(foldername);
            //find the path to folder
            //if folder doesn't exist, create one
            if(!folder.exists()) {
                folder.mkdir();
            }
            /*create a txt file for schedule data storage.
             * File name contains start of week date and username
             * so easy for system to fetch data later*/
            File file = new File(foldername + "/" + filename);
            try(FileWriter writer = new FileWriter(file)) {
                for(String[] row : result) {
                    for(String cell : row) {
                        //use " " to separate values
```

```
                        writer.write(cell + " ");
                    }
                    // next line for next row
                    writer.write("\n");
                }
                //display successful saved file
                JOptionPane.showMessageDialog(
                        null,
                        "File saved successfully",
                        "Success",
                        JOptionPane.INFORMATION_MESSAGE
                    );
                //user try and catch for exception handling
            } catch (IOException e1) {
                e1.printStackTrace();
            }

        }

    }
}
```

```java
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.text.SimpleDateFormat;
import java.time.DayOfWeek;
import java.time.LocalDate;
import java.util.Calendar;

public class Chooseweekpage extends JFrame {
    private JTable schedule;
    private DefaultTableModel tableModel;
    private JButton goback;
    private JLabel scheduleWeek;
    private User user;

    public Chooseweekpage(User user) {
        this.user = user;
        this.setDefaultCloseOperation(EXIT_ON_CLOSE); // Ensures the application closes when the window is closed
        this.setLayout(new BorderLayout()); // Sets the layout to BorderLayout
        this.setExtendedState(JFrame.MAXIMIZED_BOTH); // Maximizes the frame to cover the entire screen
        this.setTitle("Choose Week:"); // Sets the title of the window

        // Sets up the label at the top of the frame
        scheduleWeek = new JLabel("Schedule New Week", JLabel.CENTER);
        scheduleWeek.setFont(new Font("Serif", Font.BOLD, 50)); // Customizes font style and size

        // Back button to return to the previous page
        goback = new JButton("<- Back");
        goback.addActionListener(e -> {
            this.dispose(); // Closes the current frame
            new MainFrame(user); // Opens the main frame
        });

        // Panel to hold the back button and title label
        JPanel topPanel = new JPanel(new BorderLayout());
        topPanel.add(goback, BorderLayout.WEST); // Adds the
```

```
back button to the left
        topPanel.add(scheduleWeek, BorderLayout.CENTER); // Centers the title label

        // Initializes the table model with column headers
        tableModel = new DefaultTableModel(new Object[]{"Date", "Week Number", "Week A/B", "Open Schedule"}, 0);
        schedule = new JTable(tableModel);
        schedule.setRowHeight(40); // Sets the height of each row
        schedule.setDefaultEditor(Object.class, null); // Makes the table non-editable

        generateScheduleWeekData(); // Populates the table with schedule data

        // Sets custom renderers and editors for the "Open Schedule" column to use buttons
        schedule.getColumn("Open Schedule").setCellRenderer(new ButtonRenderer());
        schedule.getColumn("Open Schedule").setCellEditor(new ButtonEditor(new JCheckBox()));

        // Adds the top panel and table to the frame
        this.add(topPanel, BorderLayout.NORTH); // Adds the panel at the top
        this.add(new JScrollPane(schedule), BorderLayout.SOUTH); // Adds the table with scroll support
        this.setVisible(true); // Makes the frame visible
    }

    private void generateScheduleWeekData() {
        // Formats dates into "yyyy-MM-dd" format
        SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd");
        // Gets the start date of the next week
        Calendar calendar = Calendar.getInstance();
        LocalDate today = LocalDate.now();
        DayOfWeek dayinweek = today.getDayOfWeek(); // Gets the day of the week (e.g., Monday, Tuesday)
        int daynumber = dayinweek.getValue(); // Gets the day number (Monday = 1, ..., Sunday = 7)
```

```java
        today = today.plusDays(7); // Moves to the same day in the next week
        today = today.minusDays(daynumber - 1); // Moves to the start of that week
        int year = today.getYear();
        int month = today.getMonthValue() - 1; // Calendar uses 0-based months
        int day = today.getDayOfMonth();
        calendar.set(year, month, day); // Sets the start date in the calendar

        // Generates schedule data for the next 4 weeks
        for (int i = 1; i <= 4; i++) {
            String date = sdf.format(calendar.getTime()); // Formats the current date
            int weekNumber = i; // Week number (1 to 4)
            String abWeek = (i % 2 == 1) ? "A" : "B"; // Alternates between "A" and "B" weeks
            JButton button = new JButton("-> Schedule"); // Button for opening the schedule
            tableModel.addRow(new Object[]{date, weekNumber, abWeek, button}); // Adds a row to the table
            calendar.add(Calendar.DAY_OF_MONTH, 7); // Moves to the next week
        }
    }

    private class ButtonEditor extends DefaultCellEditor {
        /* Extends DefaultCellEditor (responsible for editing cell
         * when a user interacts with it inside a JTable)
         */
        private JButton button;
        private String date;
        private int weekNumber;
        private String weekAB;

        public ButtonEditor(JCheckBox checkBox) {
            super(checkBox);
            button = new JButton();
            button.setOpaque(true); // Ensures the button is visible
```

```java
            /* Adds ActionListener to the button to allow it to open
             * a schedule table page for the user with the selected data
             */
            button.addActionListener(e -> {
                new Scheduletablepage(date, weekNumber, weekAB, user);
            });
        }

        /* Retrieves data from the row and column of the table when a
         * button is clicked, and prepares the button component for editing
         */
        @Override
        public Component getTableCellEditorComponent(JTable table, Object value, boolean isSelected, int row, int column) {
            if (value instanceof JButton) {
                button.setText(((JButton) value).getText()); // Sets the button text
                date = (String) table.getValueAt(row, 0); // Gets the date from the table
                weekNumber = (Integer) table.getValueAt(row, 1); // Gets the week number
                weekAB = (String) table.getValueAt(row, 2); // Gets the week type (A/B)
            }
            return button; // Returns the button component
        }

        @Override
        public Object getCellEditorValue() {
            return button; // Returns the button as the editor value
        }
    }

    private static class ButtonRenderer extends JButton implements javax.swing.table.TableCellRenderer {
        // Extends TableCellRenderer to display buttons in a
```

```
table
        public ButtonRenderer() {
            setOpaque(true); // Ensures the button is visible
        }

        /* Determines which component (e.g., a button) will be displayed
         * in the table cell
         */
        @Override
        public Component getTableCellRendererComponent(JTable table, Object value, boolean isSelected, boolean hasFocus, int row, int column) {
            if (value instanceof JButton) {
                return (JButton) value; // Returns the button for rendering
            }
            return this; // Default component if not a button
        }
    }
}

```

```java
import javax.swing.*;

public class UserCurrentweekpage extends JFrame {
    private static final String[] COLUMN_HEADERS = {"Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"};
    private static final String[] ROW_HEADERS = {"Breakfast", "1st Lunch", "2nd Lunch (Downstairs)", "2nd Lunch (Stairs)", "Supper"};
    private JLabel titlelabel;
    private JTable table;
    private DefaultTableModel tablemodel;
    private String date;

    public UserCurrentweekpage(String date, User user) {
        this.date = date;
        this.setExtendedState(JFrame.MAXIMIZED_BOTH);
        this.setTitle("Schedule Table");
        this.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
        this.setLayout(new BorderLayout());

        String foldername = date + " week";
        String filename = foldername + "/" + date + " week final";
        File filefinal = new File(filename);
        //if week not scheduled
        if (!filefinal.exists()) {
            //show error
            JOptionPane.showMessageDialog(
                    this,
                    "Week not scheduled",
                    "Error",
                    JOptionPane.ERROR_MESSAGE
            );
            //end the page
            this.dispose();
            return;
        }
        //process the scheduled file
        String[][] data = processFile(filefinal);
        //label the title of the page
        titlelabel = new JLabel("Current week - Starting Date: "
```

```
+ date, JLabel.CENTER);
        titlelabel.setFont(new Font("Serif", Font.BOLD, 24));
        //table model configuration (Refer to Criterion B - User Current week page)
        tablemodel = new DefaultTableModel(data.length, COLUMN_HEADERS.length + 1) {
            //Column_Headers.length + 1 because of headers
            @Override
            public String getColumnName(int column) {
                return (column == 0) ? "Meal Time" : COLUMN_HEADERS[column - 1];
            }

            @Override
            public boolean isCellEditable(int row, int column) {
                //In this class for User, who are not allowed to edit
                //Refer to Criterion A - Success Criteria
                //disable edit for cell in the table
                return false;
            }
        };
        //enter data into JTable for visualize
        for (int i = 0; i < data.length; i++) {
            tablemodel.setValueAt(ROW_HEADERS[i], i, 0);
            for (int j = 0; j < data[i].length; j++) {
                tablemodel.setValueAt(data[i][j], i, j + 1);
            }
        }

        table = new JTable(tablemodel);
        table.setRowHeight(50);
        table.setFont(new Font("Serif", Font.PLAIN, 16));
        table.getTableHeader().setFont(new Font("Serif", Font.BOLD, 18));

        DefaultTableCellRenderer centerRenderer = new DefaultTableCellRenderer();
        centerRenderer.setHorizontalAlignment(JLabel.CENTER);
        table.setDefaultRenderer(Object.class, centerRenderer);

        // Disable selection and focus to make it fully non-
```

```java
editable
        table.setFocusable(false);
        table.setRowSelectionAllowed(false);
        table.setColumnSelectionAllowed(false);
        table.setCellSelectionEnabled(false);

        JScrollPane scrollPane = new JScrollPane(table);
        scrollPane.setPreferredSize(new Dimension(800, 600));

        this.add(titlelabel, BorderLayout.NORTH);
        this.add(scrollPane, BorderLayout.CENTER);
        this.setVisible(true);
    }

    private static String[][] processFile(File file) {
        List<String[]> lines = new ArrayList<>();
        /*use buffered reader to read the file line by line
         * and add them into the lines array that elements separated with " "*/
        //use try and catch function to handle exception
        try (BufferedReader reader = new BufferedReader(new FileReader(file))) {
            String line;
            while ((line = reader.readLine()) != null) {
                lines.add(line.trim().split("\\s+"));
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
        //return array
        return lines.toArray(new String[0][]);
    }
}
```

```java
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.time.DayOfWeek;
import java.time.LocalDate;

import javax.swing.*;

public class ChooseWeekViewPage extends JFrame implements ActionListener {
    // Declare class variables
    private User user;
    private JButton currentWeekButton;
    private JButton nextWeekButton;
    private JButton goBackButton;
    private JPanel buttonpanel;

    // Constructor: Set up the frame and UI components
    public ChooseWeekViewPage(User user) {
        this.user = user;
        this.setDefaultCloseOperation(EXIT_ON_CLOSE); // Exit the application when closing
        this.setLayout(new BorderLayout()); // Set layout as BorderLayout
        this.setExtendedState(JFrame.MAXIMIZED_BOTH); // Maximize the window
        this.setTitle("Choose Week"); // Set frame title

        // Title label setup
        JLabel titleLabel = new JLabel("Choose a Week to View", JLabel.CENTER);
        titleLabel.setFont(new Font("Serif", Font.BOLD, 24));

        // Go Back button setup
        goBackButton = new JButton("Go Back");
        goBackButton.setFont(new Font("Arial", Font.PLAIN, 16));
        goBackButton.addActionListener(this);

        // Top panel for title and "Go Back" button
        JPanel topPanel = new JPanel(new BorderLayout());
        topPanel.add(goBackButton, BorderLayout.WEST);
        topPanel.add(titleLabel, BorderLayout.CENTER);
```

```java

        // Current Week button setup
        currentWeekButton = new JButton("Current Week");
        currentWeekButton.setFont(new Font("Arial", Font.BOLD, 18));
        currentWeekButton.setPreferredSize(new Dimension(200, 80));
        currentWeekButton.addActionListener(this);

        // Next Week button setup
        nextWeekButton = new JButton("Next Week");
        nextWeekButton.setFont(new Font("Arial", Font.BOLD, 18));
        nextWeekButton.setPreferredSize(new Dimension(200, 80));
        nextWeekButton.addActionListener(this);

        // Button panel for week selection buttons
        buttonpanel = new JPanel();
        buttonpanel.add(currentWeekButton);
        buttonpanel.add(nextWeekButton);

        // Add components to the frame
        this.add(topPanel, BorderLayout.NORTH); // Add top panel to the frame
        this.add(buttonpanel, BorderLayout.CENTER); // Add button panel to the center

        this.setVisible(true); // Make the frame visible
    }

    // Handle button actions
    @Override
    public void actionPerformed(ActionEvent e) {
        if (e.getSource() == goBackButton) {
            this.dispose(); // Close the current frame
        }
        if (e.getSource() == currentWeekButton) {
            // Use java.time to calculate the start of the current week
            LocalDate today = LocalDate.now();
            DayOfWeek dayinweek = today.getDayOfWeek();
            int daynumber = dayinweek.getValue();
            LocalDate startofthisweek = today.minusDays(daynumber
```

```
- 1);
            String Startofcurrentweekdate = startofthisweek.toString();
            new UserCurrentweekpage(Startofcurrentweekdate, user);
        }
        if (e.getSource() == nextWeekButton) {
            // Use java.time to calculate the start of the next week
            LocalDate today = LocalDate.now();
            DayOfWeek dayinweek = today.getDayOfWeek();
            int daynumber = dayinweek.getValue();
            LocalDate startofnextweek = today.plusDays(7);
            startofnextweek = startofnextweek.minusDays(daynumber - 1);
            String Startofcurrentweekdate = startofnextweek.toString();
            new UserCurrentweekpage(Startofcurrentweekdate, user);
        }
    }
}
```

```java
import java.awt.BorderLayout;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.awt.Font;
import java.awt.GridBagConstraints;
import java.awt.GridBagLayout;
import java.awt.GridLayout;
import java.awt.Insets;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

import javax.swing.BorderFactory;
import javax.swing.Box;
import javax.swing.BoxLayout;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;

public class MainFrame extends JFrame implements ActionListener {

    // Declare class variables and components
    User user;
    JButton Schedule_new_week;
    JButton View_current_week_schedule;
    JButton Personal_detail;
    JLabel label;
    JPanel buttonpanel;
    JPanel southPanel;

    // MainFrame constructor: Set up the frame
    MainFrame(User user) {
        this.setDefaultCloseOperation(EXIT_ON_CLOSE); // Exit on close
        this.setLayout(new BorderLayout()); // Set BorderLayout
        this.setExtendedState(JFrame.MAXIMIZED_BOTH); // Maximize frame
        this.setTitle("Welcome"); // Frame title

        this.user = user; // Initialize user
        String text = user.getEmail(); // Get user's email

```

```java
        // Create a welcome label
        label = new JLabel("Welcome! " + text, JLabel.CENTER);
        label.setFont(new Font("Serif", Font.BOLD, 50));
        label.setBorder(BorderFactory.createEmptyBorder(100, 0, 50, 0)); // Add padding

        // Set up southPanel with GridBagLayout
        southPanel = new JPanel(new GridBagLayout());
        GridBagConstraints gbc = new GridBagConstraints();
        gbc.insets = new Insets(10, 10, 10, 10); // Add spacing between components

        // Create button panel with GridLayout
        buttonpanel = new JPanel(new GridLayout(1, 3, 20, 0));
        buttonpanel.setPreferredSize(new Dimension(900, 100)); // Set button panel size

        Dimension buttonsize = new Dimension(250, 80); // Standard button size

        // Create "Schedule new week" button
        Schedule_new_week = new JButton("Schedule new week");
        Schedule_new_week.addActionListener(this);
        Schedule_new_week.setPreferredSize(buttonsize);

        // Create "View current week schedule" button
        View_current_week_schedule = new JButton("View current week schedule");
        View_current_week_schedule.addActionListener(this);
        View_current_week_schedule.setPreferredSize(buttonsize);

        // Add buttons to the button panel
        buttonpanel.add(Schedule_new_week);
        buttonpanel.add(View_current_week_schedule);
        southPanel.add(buttonpanel, gbc); // Add button panel to south panel

        // Add components to the frame
        this.add(label, BorderLayout.NORTH); // Add welcome label to the top
        this.add(southPanel, BorderLayout.CENTER); // Add button panel to the center
```

```
        this.setVisible(true); // Make the frame visible
    }

    // Handle button actions
    @Override
    public void actionPerformed(ActionEvent e) {
        if (e.getSource() == Schedule_new_week) {
            new Chooseweekpage(user); // Open "Choose week" page
            this.dispose(); // Close current frame
        }
        if (e.getSource() == View_current_week_schedule) {
            new ChooseWeekViewPage(user); // Open "View current week schedule" page
        }
    }
}

```

```java
import java.awt.BorderLayout;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.awt.Font;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;

import javax.swing.BorderFactory;
import javax.swing.Box;
import javax.swing.BoxLayout;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JPasswordField;
import javax.swing.JTextField;
import javax.swing.border.EmptyBorder;

// Frame for changing password
public class ChangePasswordPage extends JFrame implements ActionListener {
    // File paths for user email and password storage
    private static final String Emaildocumentpath = "UserEmail.txt";
    private static final String Passworddocumentpath = "password.txt";

    // Components for the JFrame
    JLabel ForgetPassword;
    JLabel Emaillabel;
    JLabel oldPasswordLabel;
    JLabel newPasswordLabel;
    JButton resetpasswordbutton;
    JTextField Email;
    JPasswordField oldPasswordField;
    JPasswordField newPasswordField;
```

```java
    JPanel textfieldpanel;
    JPanel SouthPanel;

    // Constructor to initialize the frame
    ChangePasswordPage() {
        this.setDefaultCloseOperation(EXIT_ON_CLOSE); // Set close operation
        this.setLayout(new BorderLayout()); // Set layout manager
        this.setExtendedState(JFrame.MAXIMIZED_BOTH); // Maximize frame
        this.setTitle("Forget Password"); // Set frame title

        // Panel for buttons at the bottom
        SouthPanel = new JPanel();
        SouthPanel.setLayout(new FlowLayout(FlowLayout.CENTER, 10, 150));

        // Label for "Change Password" title
        ForgetPassword = new JLabel("Change Password", JLabel.CENTER);
        ForgetPassword.setFont(new Font("Serif", Font.BOLD, 50));
        ForgetPassword.setBorder(BorderFactory.createEmptyBorder(100, 0, 0, 0));

        // Panel for text fields
        textfieldpanel = new JPanel();
        textfieldpanel.setLayout(new BoxLayout(textfieldpanel, BoxLayout.Y_AXIS));
        textfieldpanel.setBorder(new EmptyBorder(100, 200, 20, 100));

        // Labels and fields for email, old password, and new password
        Emaillabel = new JLabel("Email: ");
        Emaillabel.setPreferredSize(new Dimension(100, 40));
        oldPasswordLabel = new JLabel("Old Password: ");
        oldPasswordLabel.setPreferredSize(new Dimension(100, 40));
        newPasswordLabel = new JLabel("New Password: ");
        newPasswordLabel.setPreferredSize(new Dimension(100,
```

```
40));

        // Button to reset password
        resetpasswordbutton = new JButton("Reset Password");
        resetpasswordbutton.setLayout(new FlowLayout(FlowLayout.CENTER));
        resetpasswordbutton.setPreferredSize(new Dimension(400, 80));
        resetpasswordbutton.addActionListener(this); // Action listener for button

        // Text fields for user input
        Email = new JTextField();
        Email.setPreferredSize(new Dimension(2000, 50));
        Email.setMaximumSize(new Dimension(2000, 50));

        oldPasswordField = new JPasswordField();
        oldPasswordField.setEchoChar('*');
        oldPasswordField.setPreferredSize(new Dimension(2000, 50));
        oldPasswordField.setMaximumSize(new Dimension(2000, 50));

        newPasswordField = new JPasswordField();
        newPasswordField.setEchoChar('*');
        newPasswordField.setPreferredSize(new Dimension(2000, 50));
        newPasswordField.setMaximumSize(new Dimension(2000, 50));

        // Add components to the text field panel
        textfieldpanel.add(Emaillabel);
        textfieldpanel.add(Box.createVerticalStrut(5));
        textfieldpanel.add(Email);
        textfieldpanel.add(Box.createVerticalStrut(20));
        textfieldpanel.add(oldPasswordLabel);
        textfieldpanel.add(Box.createVerticalStrut(5));
        textfieldpanel.add(oldPasswordField);
        textfieldpanel.add(Box.createVerticalStrut(20));
        textfieldpanel.add(newPasswordLabel);
        textfieldpanel.add(Box.createVerticalStrut(5));
        textfieldpanel.add(newPasswordField);
```

```java

        // Add components to the frame
        SouthPanel.add(resetpasswordbutton);
        this.add(textfieldpanel, BorderLayout.CENTER);
        this.add(ForgetPassword, BorderLayout.NORTH);
        this.add(SouthPanel, BorderLayout.SOUTH);
        this.setVisible(true); // Make frame visible
    }

    // Action performed method for button click
    @Override
    public void actionPerformed(ActionEvent e) {
        if (e.getSource() == resetpasswordbutton) { // Check which button was clicked
            String resetEmail = Email.getText(); // Get email input
            String oldPassword = new String (oldPasswordField.getPassword()); // Get old password input
            String newPassword = new String (newPasswordField.getPassword()); // Get new password input

            // Retrieve stored emails and passwords from files
            List<String> emails = getList(Emaildocumentpath);
            List<String> passwords = getList (Passworddocumentpath);

            // Find index of email in list
            int index = emails.indexOf(resetEmail);
            if (index == -1) { // If email not found
                JOptionPane.showMessageDialog(this, "Email not found.", "Error", JOptionPane.ERROR_MESSAGE);
                return;
            }
            if (!passwords.get(index).equals(oldPassword)) { // If old password doesn't match stored password
                JOptionPane.showMessageDialog(this, "Incorrect old password.", "Error", JOptionPane.ERROR_MESSAGE);
                return;
            }

            // Update password for the user
            List<User> userlist = User.getuserlist();
```

```java
            boolean found = false;
            for (User user : userlist) {
                if (user.getEmail().equals(resetEmail)) {
                    found = true;
                    user.changepassword(newPassword); // Change user's password
                    break;
                }
            }

            // Show success or error message based on password change
            if (found) {
                JOptionPane.showMessageDialog(this, "Your password has been changed successfully!", "Success", JOptionPane.INFORMATION_MESSAGE);
                this.dispose(); // Close current frame
                new Welcome(); // Open new welcome frame
            } else {
                JOptionPane.showMessageDialog(this, "An error occurred while changing your password.", "Error", JOptionPane.ERROR_MESSAGE);
            }
        }
    }

    // Method to read lines from file and return as list
    private List<String> getList(String path) {
        List<String> list = new ArrayList<>();
        try (BufferedReader reader = new BufferedReader(new FileReader(path))) {
            String line;
            while ((line = reader.readLine()) != null) {
                String[] items = line.split("\\|");
                for (String item : items) {
                    if (!item.trim().isEmpty()) {
                        list.add(item.trim());
                    }
                }
            }
        } catch (IOException e) {
            JOptionPane.showMessageDialog(this, "Error reading
```

```
   file: " + path, "File Error", JOptionPane.ERROR_MESSAGE);
        }
        return list;
    }
}
```

```java
import java.awt.BorderLayout;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.awt.Font;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;

import javax.swing.BorderFactory;
import javax.swing.Box;
import javax.swing.BoxLayout;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JPasswordField;
import javax.swing.JTextField;
import javax.swing.border.EmptyBorder;

//register page for user
public class RegisterPage extends JFrame implements ActionListener {
    //extends JFrame to display frame
    //extends ActionListener for implementation of buttons

    //define document path
    private static final String Emaildocumentpath = "UserEmail.txt";
    private static final String Passworddocumentpath = "password.txt";

    //initialize frame components
    JLabel Register;
    JLabel Emaillabel;
    JLabel codelabel;
    JButton Registerbutton;
    JTextField Email;
```

```java
    JPasswordField PasswordEnter;
    JPanel textfieldpanel;
    JPanel SouthPanel;

    RegisterPage() {
        //initialize default options for frame
        this.setDefaultCloseOperation(EXIT_ON_CLOSE);
        this.setLayout(new BorderLayout());
        this.setExtendedState(JFrame.MAXIMIZED_BOTH);
        this.setTitle("Register");

        //panel for register button
        SouthPanel = new JPanel();
        SouthPanel.setLayout(new FlowLayout(FlowLayout.CENTER, 10, 150));

        //Big register label
        Register = new JLabel("Register", JLabel.CENTER);
        Register.setFont(new Font("Serif", Font.BOLD, 50));
        Register.setBorder(BorderFactory.createEmptyBorder(100, 0, 0, 0));

        //user input panel
        textfieldpanel = new JPanel();
        textfieldpanel.setLayout(new BoxLayout(textfieldpanel, BoxLayout.Y_AXIS));
        textfieldpanel.setBorder(new EmptyBorder(100, 200, 20, 100));

        //label to give user instructions
        Emaillabel = new JLabel("Email: ");
        Emaillabel.setPreferredSize(new Dimension(100, 40));
        codelabel = new JLabel("Password: ");
        codelabel.setPreferredSize(new Dimension(100, 40));

        //register button configuration
        Registerbutton = new JButton("Register");
        Registerbutton.setLayout(new FlowLayout(FlowLayout.CENTER));
        Registerbutton.setPreferredSize(new Dimension(400, 80));
        Registerbutton.addActionListener(this);

```

```java
        //textfield for email/username enter
        Email = new JTextField();
        Email.setPreferredSize(new Dimension(2000, 50));
        Email.setMaximumSize(new Dimension(2000, 50));

        //password field configuration
        PasswordEnter = new JPasswordField();
        PasswordEnter.setEchoChar('*');
        PasswordEnter.setPreferredSize(new Dimension(2000, 50));
        PasswordEnter.setMaximumSize(new Dimension(2000, 50));

        //add labels and textfield/passwordfield to textfieldpanel
        textfieldpanel.add(Emaillabel);
        textfieldpanel.add(Box.createVerticalStrut(5));
        textfieldpanel.add(Email);
        textfieldpanel.add(Box.createVerticalStrut(20));
        textfieldpanel.add(codelabel);
        textfieldpanel.add(Box.createVerticalStrut(5));
        textfieldpanel.add(PasswordEnter);

        //register button added
        SouthPanel.add(Registerbutton);

        //add all panel into frame
        this.add(textfieldpanel, BorderLayout.CENTER);
        this.add(Register, BorderLayout.NORTH);
        this.add(SouthPanel, BorderLayout.SOUTH);
        //set visible
        this.setVisible(true);
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        if (e.getSource() == Registerbutton) {
            String registerEmail = Email.getText().trim();
            char[] passwordchararray = PasswordEnter.getPassword();
            String password = new String(passwordchararray);
            //check if email is already registered in the data
            if (isEmailRegistered(registerEmail)) {
                //If registered display frame of error message
```

```java
                JOptionPane.showMessageDialog(this,
                        "This email is already registered!", "Error", JOptionPane.ERROR_MESSAGE);
            } else {
                //use Buffered Writer to write the register email and password
                try (BufferedWriter emailWriter = new BufferedWriter(new FileWriter(Emaildocumentpath, true))) {
                    emailWriter.write(registerEmail + "|");
                    emailWriter.newLine();
                } catch (IOException e1) {
                    e1.printStackTrace();
                }
                try (BufferedWriter passwordWriter = new BufferedWriter(new FileWriter(Passworddocumentpath, true))) {
                    passwordWriter.write(password + "|");
                    passwordWriter.newLine();
                } catch (IOException e1) {
                    e1.printStackTrace();
                }
                //close Register frame
                this.dispose();
                //back to Welcome frame
                new User(registerEmail, password);
                new Welcome();
            }
        }
    }

    //check if email is registered
    private boolean isEmailRegistered(String email) {
        //user buffered Reader to loop through the email to check repetition
        //user try and catch to handle exceptions
        try (BufferedReader reader = new BufferedReader(new FileReader(Emaildocumentpath))) {
            String line;
            while ((line = reader.readLine()) != null) {
                if (line.contains(email + "|")) {
                    return true;
                }
            }
```

```
        } catch (IOException e) {
            e.printStackTrace();
        }
        return false;
    }
}

```

```java
import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.awt.Font;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;

import javax.swing.BorderFactory;
import javax.swing.Box;
import javax.swing.BoxLayout;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JPasswordField;
import javax.swing.JTextField;
import javax.swing.border.EmptyBorder;

public class Welcome extends JFrame implements ActionListener{
    //extends JFrame to display frame
    //extends ActionListener for implementation of buttons

    //define document path
    private static final String Emaildocumentpath = "UserEmail.txt";
    private static final String Passworddocumentpath = "password.txt";
    private static final String AdminEmaildocumentpath = "AdminEmail.txt";
    private static final String AdminPasworddocumentpath = "Adminpassword.txt";

    //define key components for frame
    JPanel SouthPanel;
    JPanel Loginbutton;
```

```java
    JPanel textfield;
    JPanel buttons;
    JLabel Welcome;
    JLabel Emaillabel;
    JLabel Passwordlabel;
    JTextField Email;
    JPasswordField PasswordEnter;
    JButton ForgotPassword;
    JButton Register;
    JButton Login;
    //user input
    String INPUT;
    String Password;

    public Welcome() {
        //store user and admin into Array list
        List<String> useremail = getList(Emaildocumentpath);
        List<String> password = getList(Passworddocumentpath);


        for(int i = 0;i<useremail.size();i++) {
            new User(useremail.get(i),password.get(i));
        }

        List<String> adminemail = getList(AdminEmaildocumentpath);
        List<String> adminpassword = getList(AdminPassworddocumentpath);
        for(int i = 0;i<adminemail.size();i++) {
            new Admin(adminemail.get(i), adminpassword.get(i));
        }
        //Frame default options
        this.setDefaultCloseOperation(EXIT_ON_CLOSE);
        this.setLayout(new BorderLayout());
        this.setExtendedState(JFrame.MAXIMIZED_BOTH);
        this.setTitle("Welcome!");

        //welcome label on top of the frame
        Welcome = new JLabel("Welcome!", JLabel.CENTER);
        Welcome.setFont(new Font("Serif", Font.BOLD, 50));
        Welcome.setBorder(BorderFactory.createEmptyBorder(100, 0, 0, 0));
```

```java

        //a Panel for user input(named textfield)
        textfield = new JPanel();
        textfield.setLayout(new BoxLayout(textfield, BoxLayout.Y_AXIS));
        textfield.setBorder(new EmptyBorder(100, 200, 20, 100));

        //Panel for login button
        Loginbutton = new JPanel();
        Loginbutton.setLayout(new FlowLayout(FlowLayout.CENTER,10,40));
        Loginbutton.setPreferredSize(new Dimension(200,300));

        //panel for buttons
        buttons = new JPanel();
        buttons.setLayout(new FlowLayout(FlowLayout.CENTER,10,50));
        buttons.setPreferredSize(new Dimension(200,150));

        //Label of email/username and password to give user instructions
        Emaillabel = new JLabel("Email: ");
        Emaillabel.setPreferredSize(new Dimension(100,40));
        Passwordlabel = new JLabel("Password: ");
        Passwordlabel.setPreferredSize(new Dimension(100,40));

        //Register button and configuration
        Register = new JButton("Register");
        Register.setFont(new Font("Serif", Font.BOLD, 20));
        Register.setForeground(Color.BLACK);
        //invisible border
        Register.setBorderPainted(false);
        Register.setContentAreaFilled(false);
        //enable instruction after button is clicked
        Register.addActionListener(this);

        //Change Password button and configuration
        ForgotPassword = new JButton("Change password");
        ForgotPassword.setFont(new Font("Serif", Font.BOLD, 20));
        ForgotPassword.setForeground(Color.BLACK);
        ForgotPassword.setBorderPainted(false);
```

```
        ForgotPassword.setContentAreaFilled(false);
        //enable instruction after button is clicked
        ForgotPassword.addActionListener(this);

        //button for login
        Login = new JButton("Login");
        Login.setLayout(new FlowLayout(FlowLayout.CENTER));
        Login.setPreferredSize(new Dimension(400,80));
        Login.addActionListener(this);

        //JPasswordField for password enter
        PasswordEnter = new JPasswordField();
        //keep privacy when user is entering password
        PasswordEnter.setEchoChar('*');
        PasswordEnter.setPreferredSize(new Dimension(2000, 50));
        PasswordEnter.setMaximumSize(new Dimension(2000, 50));

        //textfield for Email enter
        Email = new JTextField();
        Email.setPreferredSize(new Dimension(2000, 50));
        Email.setMaximumSize(new Dimension(2000, 50));

        //south panel for buttons
        SouthPanel = new JPanel();
        SouthPanel.setLayout(new BoxLayout(SouthPanel, BoxLayout.Y_AXIS));

        textfield.add(Emaillabel);
        //distance between label and textfield
        textfield.add(Box.createVerticalStrut(5));
        textfield.add(Email);
        textfield.add(Box.createVerticalStrut(10));
        textfield.add(Passwordlabel);
        textfield.add(Box.createVerticalStrut(5));
        textfield.add(PasswordEnter);
        //add buttons to panels
        buttons.add(Register);
        buttons.add(ForgotPassword);
        Loginbutton.add(Login);
        SouthPanel.add(Loginbutton);
        SouthPanel.add(buttons);
        //set location for panels
```

```
        this.add(Welcome, BorderLayout.NORTH);
        this.add(textfield, BorderLayout.CENTER);
        this.add(SouthPanel, BorderLayout.SOUTH);
        this.setVisible(true);
    }

    @Override //action performed after button is clicked
    public void actionPerformed(ActionEvent e) {
        if(e.getSource() == ForgotPassword) {
            //when a change password button is clicked by user
            this.dispose();
            //close current page and open the change password page
            new ChangePasswordPage();
        }
        else if(e.getSource() == Register) {
            //when register button is clicked
            //close current page and open a Register page
            this.dispose();
            new RegisterPage();
        }
        else if(e.getSource() == Login) {
            //get user input (use character array to keep the password textfield private)
            char[] passwordchararray = PasswordEnter.getPassword();
            String password = new String(passwordchararray);
            INPUT = Email.getText();
            //check if login detail accepted (refer to figure 1.1 check() method)
            boolean valid = check(AdminEmaildocumentpath, AdminPasworddocumentpath, password, INPUT)
                    || check(Emaildocumentpath, Passworddocumentpath, password, INPUT);
            if(valid) {
                //differentiate between User and Admin
                User currentuser = User.getuser(INPUT);
                if(currentuser.isAdmin()) {
                    //open an Admin Main Frame
                    Admin currentadmin = Admin.getadmin(INPUT);
                    new AdminMainFrame(currentadmin);
                }
```

```
                else {
                    //open an User Main Frame
                    new MainFrame(currentuser);
                }
                //close Welcome Page
                this.dispose();
            }
            else {
                //display input error
                JOptionPane.showOptionDialog(
                        null,
                        "incorrect email or password",
                        "error",
                        JOptionPane.DEFAULT_OPTION,
                        JOptionPane.ERROR_MESSAGE,
                        null,
                        new Object[] { "OK" },
                        "OK"
                    );
            }
        }

    }

    private List<String> getList(String path) {
        //User dynamic data structure Array List to append data
        List<String> list = new ArrayList<>();
        //user buffered reader from java.io to extract data from
txt file
        //user try and catch to handle exception
        try (BufferedReader reader = new BufferedReader(new
FileReader(path))) {
            String line;
            /* write user or admin Password into Array split
with the character
             "|" and handle exception*/
            while ((line = reader.readLine()) != null) {
                String[] items = line.split("\\|");
                for (String item : items) {
                    if (!item.trim().isEmpty()) {
                        list.add(item.trim());
                    }
```

```
                }
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
        return list;
    }

    public boolean check(String emailPath, String passwordPath, String passwordEntered, String emailEntered) {
        List<String> emailList = getList(emailPath);
        List<String> passwordList = getList(passwordPath);
        //check if the entered email and password match with any of the registered email and password
        for (int i = 0; i < emailList.size(); i++) {
            String storedEmail = emailList.get(i);
            String storedPassword = passwordList.get(i);
            if (emailEntered.equals(storedEmail) && passwordEntered.equals(storedPassword)) {
                return true;
            }
        }
        return false;
    }
}
```

```java
import java.util.ArrayList;
import java.util.List;

public class Admin extends User{//extends User(inheritance)

    //Admin Arraylist to store all admin
    public static List<Admin> adminlist = new ArrayList<>();

    public Admin(String Email, String password) {
        //inherit User detail
        super(Email, password);
        //add new admin to admin Arraylist
        adminlist.add(this);

    }
    //return the whole adminlist
    public static List<Admin> getAdminlist(){
        return adminlist;
    }

    //get admin from admin list
    public static Admin getadmin(String useremail){
        Admin returnadmin = null;
        for(int i =0;i<adminlist.size();i++) {
            if(adminlist.get(i).getEmail().equals(useremail)) {
                returnadmin = adminlist.get(i);
            }
        }
        return returnadmin;
    }

}

```

```java
import java.awt.Image;
import java.util.ArrayList;
import java.util.List;
public class User {//class User

    //User List to store user at the start of program
    //static own by class not individual user
    public static List<User> userlist = new ArrayList<>();
    //all user details
    private String UserID;
    private String Email;
    private String International_sign;
    private int Phone_number;
    private String Full_Name;
    private Image icon;
    private String password;

    //register with email/username and password
    public User(String Email, String password) {

        this.Email = Email;
        this.password = password;
        //add to user list
        userlist.add(this);
    }

    //assessors
    public String getUserID() {
        return this.UserID;
    }

    public String getEmail() {
        return this.Email;
    }

    public String getInternational_sign() {
        return this.International_sign;
    }

    public int Phone_number() {
        return this.Phone_number;
    }
```

```java

    public String getName() {
        return this.Full_Name;
    }

    public Image getImage() {
        return this.icon;
    }

    public String getpassword() {
        return this.password;
    }

    //mutators
    public void changepassword(String password) {
        this.password = password;
    }

    //get user from email by scanning the arraylist
    public static User getuser(String useremail){
        User returnuser = null;
        for(int i =0;i<userlist.size();i++) {
            if(userlist.get(i).getEmail().equals(useremail)) {
                returnuser = userlist.get(i);
            }
        }
        return returnuser;
    }
    //assessor for user arraylist
    public static List<User> getuserlist(){
        return userlist;
    }

    //check if that user is an admin
    public Boolean isAdmin() {
        Boolean result = false;
        //get Admin array list
        List<Admin> adminlist = Admin.getAdminlist();
        for(Admin i : adminlist) {
            if(this.equals(i)) {
                result = true;
            }
```

```
        }
        return result;

    }

}

```

```java

public class Main {
    public static void main(String[] args) {
        new Welcome(); //start program
    }
}


```